Boogle Dox — Product Requirements & Product Thinking

Project: Ajaia Full Stack Product Engineer take-home

Status: Planning document — implementation and test results not yet verified

Timebox: 4–6 hours

Why I chose this product direction

I started by asking myself what I actually rely on when I use an online document editor. For me, being able to name a document, find the familiar File, Edit, View, Insert, Format, and Tools menus, format text, undo mistakes, and share a document are all important. I wanted Boogle Dox to feel recognizable right away, without trying to rebuild every Google Docs feature.

I also wanted to think carefully about collaboration. Simultaneous editing can be useful, but I've experienced it becoming messy when multiple people change the same content at once. My idea is to eventually support controlled editing sessions: people can see when someone else is editing, view the last saved version, and get notified when editing is finished. I think that is an interesting alternative to live multi-cursor editing, but I do not want to sacrifice reliable document creation and sharing to build it during a short assignment.

Who it is for

Small teams who want to create, format, import, and share documents in a browser without installing software. For this assignment, two demo users are enough to demonstrate the ownership and sharing flows.

Primary user journey

1. Open the app and see documents I own versus documents shared with me.
2. Create a document and give it a meaningful title.
3. Enter and format content using a familiar toolbar and working menus.
4. Save, refresh the page, and reopen the same document with formatting preserved.
5. Import a TXT or Markdown file as an editable document.
6. Share a document with another account as a viewer or editor, then copy its link.
7. Switch to the recipient account and verify their allowed actions; verify an unauthorized account cannot access it.

Scope and acceptance criteria

P0 — complete these before anything else

• Document creation, renaming, saving, reopening, and deletion only if there is time.

• Rich text: bold, italic, underline, headings, bullet and numbered lists.

• Working File, Edit, View, Insert, Format, and Tools menus with a small, deliberate set of actual actions.

• Undo and redo.

• File import: TXT and MD with visible file type and size limits.

• Ownership and sharing: owner, editor, viewer; owner can grant and revoke access.

• Copy link: copies the canonical document URL but does not bypass access checks.

• Clear owned/shared sections.

• Persistence of document content, title, formatting, and share grants.

• Validation, clear error messages, at least one meaningful automated test, and a working hosted deployment.

P1 — add after the core flow is dependable

• Font family/size, text color, highlight, alignment, line spacing, indentation, checklist.

• Page-like canvas, visible margins, zoom, and a visual ruler.

• Saving/saved/error status.

• Controlled edit sessions and real-time editing-presence notices, if time permits.

Explicitly out of scope for the first submission

• Full concurrent multi-cursor collaborative editing.

• Advanced Google Docs-equivalent pagination and ruler-based tab stops.

• Comments, suggestions, revision history, and complex exports.

• Full DOCX parsing and enterprise-grade access management.

Product rules

Owner: can view, edit, rename, share, change permissions, and revoke access.

Editor: can view and edit but cannot change sharing settings.

Viewer: can read but cannot alter the document.

Unauthorized user: receives an access-denied response even if given the URL.

A copied link is an address, not a permission grant. A separate "anyone with the link" viewer feature would require an intentional access policy and a revocable token; it should not be silently enabled.

What success looks like

A reviewer can open the deployed app, create and format a document, import a file, save it, refresh, share it, and verify the difference between viewer and editor permissions in a few minutes.

Decisions and tradeoffs

I chose depth over breadth. The familiar menus matter to the experience, but showing a menu full of nonfunctional buttons would be misleading. I would rather ship fewer menu actions that actually work. I also chose access correctness and persistence ahead of live collaboration; a sharing demo isn't convincing if a viewer can modify the document or if someone can open a restricted document by guessing the URL.

Verification status

This document records my intended requirements and reasoning. Features should be marked complete only after being implemented and tested in the deployed app.